1. SJF_Non_Preemptive

```
#include <stdio.h>
#define MAX 10

struct sjf {
    int pid;
    int bt;
    int at;
    int ct;
        int wt;
        int tat;
        int done;
    };

    void input(struct sjf f[], int n) {
        for (int i = 0; i < n; i++) {
            f[i].pid = i + 1;
            printf("Enter arrival time and burst time of process %d: ", i + 1);
            scanf("%d %d", &f[i].at, &f[i].bt);
            f[i].done = 0;
        }
    }

    void calculate(struct sjf f[], int n) {
        int time = 0, completed = 0;
        float avgTat = 0, avgWt = 0;

        while (completed < n) {
            int idx = -1;
            int minBt = 1e9;
            for (int i = 0; i < n; i++) {
                if (!f[i].done && f[i].at <= time && f[i].bt < minBt) {
                    minBt = f[i].bt;
                    idx = i;
                }
            }
```

```
            if (idx == -1) {
                time++;
                continue;
            }
            time += f[idx].bt;
            f[idx].ct = time;
            f[idx].tat = f[idx].ct - f[idx].at;
            f[idx].wt = f[idx].tat - f[idx].bt;
            f[idx].done = 1;
            completed++;
            avgTat += f[idx].tat;
            avgWt += f[idx].wt;
        }

        printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
        for (int i = 0; i < n; i++) {
            printf("P%d\t%d\t%d\t%d\t%d\t%d\n", f[i].pid, f[i].at, f[i].bt, f[i].ct, f[i].tat, f[i].wt);
        }
        printf("\nAverage TAT: %.2f\n", avgTat / n);
        printf("Average WT: %.2f\n", avgWt / n);
    }

    int main() {
        struct sjf f[MAX];
        int n;
        printf("Enter number of processes: ");
        scanf("%d", &n);
        input(f, n);
        calculate(f, n);
        return 0;
    }
```

Output:

```
Enter number of processes: 5
Enter arrival time and burst time of process 1: 0
3
Enter arrival time and burst time of process 2: 2
2
Enter arrival time and burst time of process 3: 3
5
Enter arrival time and burst time of process 4: 4
4
Enter arrival time and burst time of process 5: 6
1

Process AT      BT      CT      TAT     WT
P1      0       3       3       3       0
P2      2       2       5       3       1
P3      3       5       15      12      7
P4      4       4       9       5       1
P5      6       1       10      4       3

Average TAT: 5.40
Average WT: 2.40
PS C:\Users\sruja\OneDrive\OS>
```

16.SJF_Preemptive

```
#include <stdio.h>
#define MAX 10

struct sjf {
    int pid;
    int bt;
    int at;
    int ct;
    int wt;
    int tat;
    int rt;
};

void input(struct sjf f[], int n) {
    for (int i = 0; i < n; i++) {
        f[i].pid = i + 1;
        printf("Enter arrival time and burst time of process
%d: ", i + 1);
        scanf("%d %d", &f[i].at, &f[i].bt);
        f[i].rt = f[i].bt;
    }
}

void calculate(struct sjf f[], int n) {
    int time = 0, completed = 0;
    float avgTat = 0, avgWt = 0;

    while (completed < n) {
        int idx = -1;
        int minRt = 1e9;

        for (int i = 0; i < n; i++) {
            if (f[i].rt > 0 && f[i].at <= time && f[i].rt <
minRt) {
                minRt = f[i].rt;
                idx = i;
            }
```

```
        }

        if (idx == -1) {
            time++;
            continue;
        }

        f[idx].rt--;
        time++;

        if (f[idx].rt == 0) {
            f[idx].ct = time;
            f[idx].tat = f[idx].ct - f[idx].at;
            f[idx].wt = f[idx].tat - f[idx].bt;
            avgTat += f[idx].tat;
            avgWt += f[idx].wt;
            completed++;
        }
    }

    printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
               f[i].pid, f[i].at, f[i].bt, f[i].ct,
f[i].tat, f[i].wt);
    }
    printf("\nAverage TAT: %.2f\n", avgTat / n);
    printf("Average WT: %.2f\n", avgWt / n);
}

int main() {
    struct sjf f[MAX];
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    input(f, n);
    calculate(f, n);
    return 0;
```

```
}
```

Output:

```
Enter number of processes: 5
Enter arrival time and burst time of process 1: 0
3
Enter arrival time and burst time of process 2: 2
2
Enter arrival time and burst time of process 3: 3
5
Enter arrival time and burst time of process 4: 4
4
Enter arrival time and burst time of process 5: 6
1

Process AT      BT      CT      TAT     WT
P1      0       3       3       3       0
P2      2       2       5       3       1
P3      3       5       15      12      7
P4      4       4       10      6       2
P5      6       1       7       1       0

Average TAT: 5.00
Average WT: 2.00

Process returned 0 (0x0)   execution time : 21.395 s
Press any key to continue.
```

3. Priority_Non_Preemptive

```
#include <stdio.h>
#define MAX 10

struct process {
    int pid, bt, at, pr, ct, wt, tat, done;
};

void input(struct process p[], int n) {
    for (int i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("Enter arrival time, burst time and priority
of process %d: ", i + 1);
        scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].pr);
        p[i].done = 0;
    }
}

void calculate(struct process p[], int n) {
    int time = 0, completed = 0;
    float avgTat = 0, avgWt = 0;
    while (completed < n) {
        int idx = -1, highestPr = 1e9;
        for (int i = 0; i < n; i++) {
            if (!p[i].done && p[i].at <= time && p[i].pr <
highestPr) {
                highestPr = p[i].pr;
                idx = i;
            }
        }
        if (idx == -1) {
            time++;
            continue;
        }
        time += p[idx].bt;
        p[idx].ct = time;
        p[idx].tat = p[idx].ct - p[idx].at;
        p[idx].wt = p[idx].tat - p[idx].bt;
```

```
        p[idx].done = 1;
        completed++;
        avgTat += p[idx].tat;
        avgWt += p[idx].wt;
    }
    printf("\nProcess\tAT\tBT\tPR\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n", p[i].pid,
p[i].at, p[i].bt, p[i].pr, p[i].ct, p[i].tat, p[i].wt);
    }
    printf("\nAverage TAT: %.2f\n", avgTat / n);
    printf("Average WT: %.2f\n", avgWt / n);
}

int main() {
    struct process p[MAX];
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    input(p, n);
    calculate(p, n);
    return 0;
}
```

Output:

```
Enter number of processes: 5
Enter arrival time, burst time and priority of process 1: 0
3
5
Enter arrival time, burst time and priority of process 2: 2
2
3
Enter arrival time, burst time and priority of process 3: 3
5
2
Enter arrival time, burst time and priority of process 4: 4
4
4
Enter arrival time, burst time and priority of process 5: 6
1
1

Process AT      BT      PR      CT      TAT     WT
P1      0       3       5       3       3       0
P2      2       2       3       11      9       7
P3      3       5       2       8       5       0
P4      4       4       4       15      11      7
P5      6       1       1       9       3       2

Average TAT: 6.20
Average WT: 3.20

Process returned 0 (0x0)   execution time : 43.533 s
Press any key to continue.
```

4.Priority_Preemptive

```
#include <stdio.h>
#define MAX 10

struct process {
    int pid, bt, at, pr, ct, wt, tat, rt, done;
};

void input(struct process p[], int n) {
    for (int i = 0; i < n; i++) {
        p[i].pid = i + 1;
        scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].pr);
        p[i].rt = p[i].bt;
        p[i].done = 0;
    }
}

void calculate(struct process p[], int n) {
    int time = 0, completed = 0;
    float avgTat = 0, avgWt = 0;
    while (completed < n) {
        int idx = -1, highestPr = 1e9;
        for (int i = 0; i < n; i++) {
            if (!p[i].done && p[i].at <= time && p[i].pr <
highestPr) {
                highestPr = p[i].pr;
                idx = i;
            }
        }
        if (idx == -1) {
            time++;
            continue;
        }
        p[idx].rt--;
        time++;
        if (p[idx].rt == 0) {
            p[idx].ct = time;
            p[idx].tat = p[idx].ct - p[idx].at;
```

```
            p[idx].wt = p[idx].tat - p[idx].bt;
            p[idx].done = 1;
            completed++;
            avgTat += p[idx].tat;
            avgWt += p[idx].wt;
        }
    }
    printf("\nProcess\tAT\tBT\tPR\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
               p[i].pid, p[i].at, p[i].bt, p[i].pr,
               p[i].ct, p[i].tat, p[i].wt);
    }
    printf("\nAverage TAT: %.2f\n", avgTat / n);
    printf("Average WT: %.2f\n", avgWt / n);
}

int main() {
    struct process p[MAX];
    int n;
    scanf("%d", &n);
    input(p, n);
    calculate(p, n);
    return 0;
}
```

Output:

```
Enter number of processes: 5
Enter AT BT PR for P1: 0
3
5
Enter AT BT PR for P2: 2
2
3
Enter AT BT PR for P3: 3
5
2
Enter AT BT PR for P4: 4
4
4
Enter AT BT PR for P5: 6
1
1

Process AT      BT      PR      CT      TAT     WT
P1      0       3       5       15      15      12
P2      2       2       3       10      8       6
P3      3       5       2       9       6       1
P4      4       4       4       14      10      6
P5      6       1       1       7       1       0

Average TAT: 8.00
Average WT: 5.00
```